

Python Learning Hub

Very Detailed Architecture, Folder Structure, Development,
Deployment, Routing, SEO, and Implementation Plan

September 25, 2026

Contents

1 Document Purpose

This document defines a complete implementation plan for the Python Learning Hub.

The central architectural goal is to build **one product on one domain** while using two different presentation technologies:

- **FastAPI + Jinja2 + HTMX** for public, content-oriented, search-engine-friendly pages.
- **Flutter Web** for the authenticated and highly interactive application.

FastAPI remains the **single application boundary and single public HTTP entry point**. The browser should not need to know that different pages are implemented using different frontend technologies.

The intended mental model is:

This is one FastAPI application with two presentation layers: a server-rendered web layer and a Flutter application layer.

The architecture is deliberately designed so that business logic, data access, authentication rules, and domain concepts are not duplicated merely because the presentation layer changes.

2 Project Vision

Python Learning Hub is intended to be a learning and community platform centered around Python and related technical learning material.

The platform may provide:

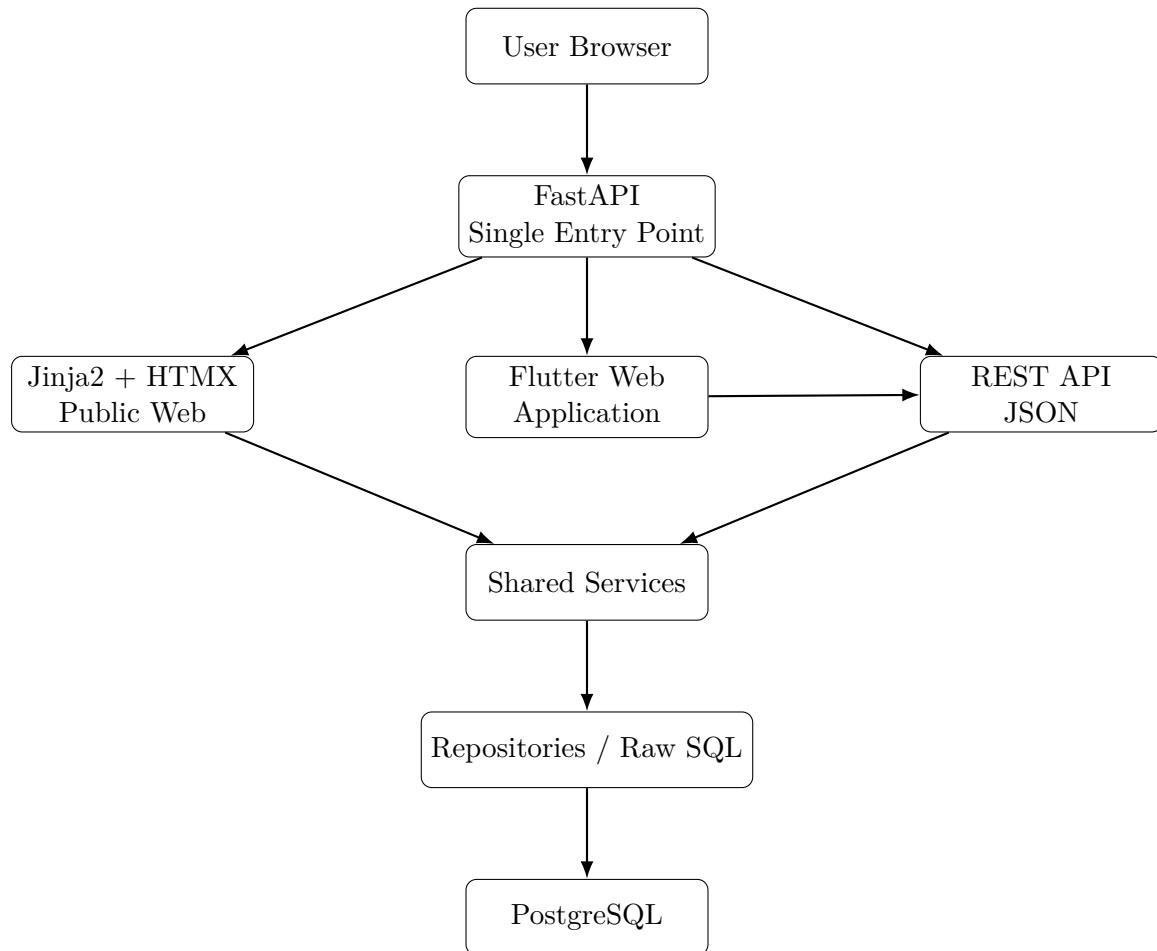
- learning roadmaps;
- topics and categories;
- tutorials and guides;
- searchable educational content;
- code examples;
- learner progress tracking;
- bookmarks;
- user profiles;
- public discussions;
- authenticated discussions and participation;
- administrative content management;
- technical SEO;
- REST API access;
- real-time features through WebSockets where required.

The most important architectural requirement is that public educational content should remain usable and discoverable without requiring the visitor to run a large client-side application.

At the same time, authenticated workflows can benefit from Flutter's application-style UI.

3 Core Architectural Principle

The system is divided into three conceptual worlds.



The three conceptual worlds are:

1. **Public Web World:** Jinja2 and HTMX.
2. **Application World:** Flutter Web under `/app/`.
3. **Backend World:** FastAPI REST endpoints under `/api/v1/` and WebSockets under `/ws/`.

They share the same backend services and data-access layer.

4 Why Use Two Presentation Layers?

A single frontend technology is not required to serve every page equally well.

4.1 Public Content

Public pages such as:

- roadmap pages;
- topic pages;
- tutorials;
- guides;

- public search results;
- public discussions;

benefit from ordinary HTML being available directly from the server.

A search engine crawler should be able to request:

/topics/python-functions

and receive meaningful HTML containing headings, content, links, metadata, and structured information.

Therefore these pages use Jinja2.

4.2 Application Workflows

Authenticated pages such as:

- dashboard;
- personal profile;
- progress;
- bookmarks;
- interactive learner tools;
- administration;

can be implemented as a Flutter Web application.

These pages are application-oriented rather than primarily search-oriented.

5 One Domain

The production system should expose one main domain, for example:

<https://pythonlearninghub.com>

The visitor should not have to think about separate frontend servers.

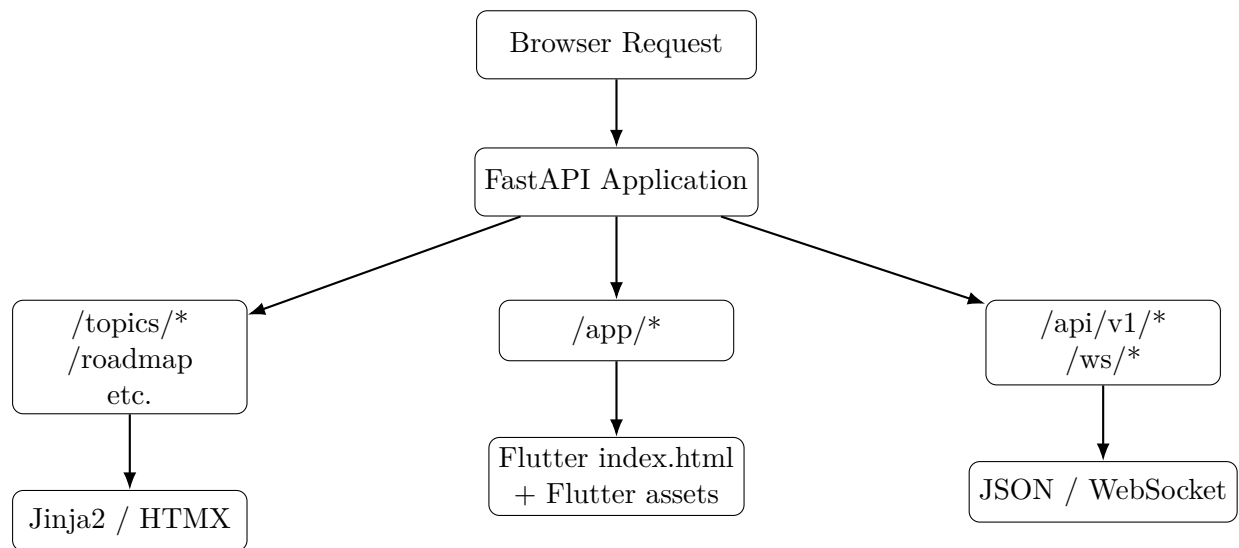
The URL structure is:

URL Pattern	Technology	Purpose
/	Jinja2	Home page
/roadmap	Jinja2	Public roadmap
/topics/*	Jinja2	Public topic pages
/tutorials/*	Jinja2	Public tutorials
/guides/*	Jinja2	Public guides
/search	Jinja2	Public/search-oriented results
/about	Jinja2	Public information
/web-components/*	HTMX/Jinja2	HTML fragments
/app	Flutter	Application entry
/app/*	Flutter	Application screens
/api/v1/*	FastAPI JSON	Flutter/backend API
/ws/*	FastAPI WebSocket	Real-time features

This URL organization is one of the most important design decisions in the project.

6 Exact Request Routing Model

Every request first reaches the FastAPI deployment.



FastAPI is therefore responsible for deciding which presentation mechanism should handle a request.

7 Recommended Repository Structure

The recommended repository keeps the Python backend and Flutter source code conceptually separate.

```

python-learning-hub/
|
+-- backend/
|   |
|   +-- app/
|       |
|       +-- main.py
|       |
|       +-- routers/
|           |
|           +-- web/
|               |
|               +-- pages/
|                   +-- home.py
|                   +-- roadmap.py
|                   +-- topics.py
|                   +-- tutorials.py
|                   +-- search.py
|               |
|               +-- components/
|                   +-- search.py
|                   +-- progress.py
|                   +-- comments.py
|           |
|           +-- api/
|               +-- v1/
  
```

```

| | | +-- auth.py
| | | +-- users.py
| | | +-- topics.py
| | | +-- progress.py
| | | +-- bookmarks.py
| | | +-- discussions.py
| | |
| | +-- websocket/
| |     +-- chat.py
| |
| +-- services/
| |     +-- auth_service.py
| |     +-- topic_service.py
| |     +-- progress_service.py
| |     +-- bookmark_service.py
| |     +-- discussion_service.py
| |
| +-- repositories/
| |     +-- user_repository.py
| |     +-- topic_repository.py
| |     +-- progress_repository.py
| |     +-- bookmark_repository.py
| |     +-- discussion_repository.py
| |
| +-- schemas/
| |     +-- auth.py
| |     +-- user.py
| |     +-- topic.py
| |     +-- progress.py
| |     +-- bookmark.py
| |     +-- discussion.py
| |
| +-- templates/
| |     +-- base.html
| |     +-- home.html
| |     +-- roadmap.html
| |     +-- topic.html
| |     +-- tutorial.html
| |     +-- search.html
| |     +-- components/
| |
| +-- static/
| |     +-- css/
| |     +-- js/
| |     +-- images/
| |
| +-- flutter/
| |     +-- web/
| |         +-- index.html
| |         +-- flutter.js
| |         +-- main.dart.js
| |         +-- assets/
| |         +-- canvaskit/
| |         +-- ...
| |
| +-- pyproject.toml
|
+-- frontend/

```

```
| |
| +-- flutter_app/
|   +-- lib/
|   +-- assets/
|   +-- test/
|   +-- pubspec.yaml
|   +-- analysis_options.yaml
|   +-- ...
|
+-- scripts/
|   +-- build_flutter.py
|
+-- README.md
```

7.1 Important Distinction

The directory:

```
frontend/flutter_app/
```

contains the **Flutter source project**.

The directory:

```
backend/app/flutter/web/
```

contains the **compiled Flutter Web deployment artifact**.

Do not confuse these two.

8 Responsibilities of Each Backend Layer

8.1 Routers

Routers decide:

- which URL is being requested;
- which authentication dependency applies;
- which service should execute;
- what type of response should be returned.

Routers should not contain large amounts of business logic.

8.2 Services

Services contain application/business operations.

For example:

```
get_topic_by_slug()
create_bookmark()
update_progress()
authenticate_user()
search_topics()
```

The same service can be called by:

- a Jinja2 page;
- an HTMX endpoint;

- a REST API endpoint;
- a WebSocket workflow where appropriate.

8.3 Repositories

Repositories contain database access.

For a raw-SQL approach, a repository may execute parameterized SQL using the project's async database mechanism.

The important dependency direction is:

```
Router
|
v
Service
|
v
Repository
|
v
PostgreSQL
```

Not:

```
Router
|
+--> SQL
+--> business rules
+--> authentication logic
+--> HTML rendering
```

The second approach would make the project difficult to maintain.

9 Jinja2 and HTMX Architecture

Jinja2 is responsible for complete public pages.

Example:

```
/topics/python-functions
```

returns a complete HTML document.

HTMX is responsible for smaller interactions inside that page.

Example:

```
/web-components/topics/python-functions/progress
```

may return only:

```
<div id="progress">
...
</div>
```

The rule is:

HTMX changes parts of the Jinja2 web world; it does not become the navigation mechanism for the Flutter application.

10 Flutter Web Architecture

The Flutter project lives under:

```
frontend/flutter_app/
```

Its internal structure can follow normal Flutter conventions:

```
flutter_app/
|
+-- lib/
|   +-- main.dart
|   +-- app/
|       +-- app.dart
|       +-- router.dart
|       |
|       +-- features/
|           +-- auth/
|           +-- dashboard/
|           +-- profile/
|           +-- progress/
|           +-- bookmarks/
|           +-- admin/
|           |
|       +-- core/
|           +-- api/
|           +-- models/
|           +-- storage/
|           +-- widgets/
|
+-- assets/
+-- test/
+-- pubspec.yaml
```

Flutter is responsible for application-level navigation such as:

```
/app
/app/dashboard
/app/profile
/app/progress
/app/bookmarks
/app/admin
```

11 The Boundary Between Flutter and Jinja2

The boundary is the URL.

Suppose a user is viewing:

```
/app/dashboard
```

and clicks a link to a public tutorial:

```
/topics/python-asyncio
```

The browser leaves the Flutter application and performs normal navigation.

The request reaches FastAPI again.

FastAPI sees:

```
/topics/python-asyncio
```

and returns Jinja2 HTML.

Conversely, a Jinja2 page can contain:

```
<a href="/app/dashboard">Dashboard</a>
```

This sends the browser into the Flutter application.

The two worlds are therefore connected by normal URLs.

12 Detailed Request Flows

12.1 Public Page Request

```
Browser
|
| GET /topics/python
v
FastAPI
|
v
web/pages/topics.py
|
v
TopicService
|
v
TopicRepository
|
v
PostgreSQL
|
v
Jinja2 topic.html
|
v
HTML response
|
v
Browser
```

12.2 HTMX Request

```
Browser
|
| GET /web-components/search?q=async
v
FastAPI
|
v
web/components/search.py
|
v
SearchService
|
v
SearchRepository
|
```

```
v
Jinja2 partial
|
v
HTMX swaps HTML fragment
```

12.3 Flutter Page Request

```
Browser
|
| GET /app/dashboard
v
FastAPI
|
v
Flutter frontend handler
|
v
Flutter index.html
|
v
Flutter JavaScript/assets
|
v
Flutter Router
|
v
Dashboard screen
```

12.4 Flutter API Request

```
Flutter
|
| GET /api/v1/progress
v
FastAPI
|
v
api/v1/progress.py
|
v
ProgressService
|
v
ProgressRepository
|
v
PostgreSQL
|
v
JSON
|
v
Flutter
```

13 FastAPI Application Composition

The conceptual structure of `main.py` is:

```
from fastapi import FastAPI

app = FastAPI()

# Web pages
app.include_router(web_pages_router)

# HTMX components
app.include_router(web_components_router)

# REST API
app.include_router(api_v1_router)

# WebSockets
app.include_router(websocket_router)

# Flutter Web
# /app/* is handled by the Flutter frontend
```

The actual Flutter handler should be implemented carefully so that:

- static Flutter assets are served;
- `/app/` loads Flutter;
- deep links such as `/app/dashboard` also load Flutter's entry document;
- public routes are not accidentally swallowed by the Flutter fallback.

A critical rule is:

Never create a global catch-all Flutter fallback for every URL.

If every unknown URL returned Flutter's `index.html`, a public URL such as:

```
/topics/python
```

could incorrectly become a Flutter page.

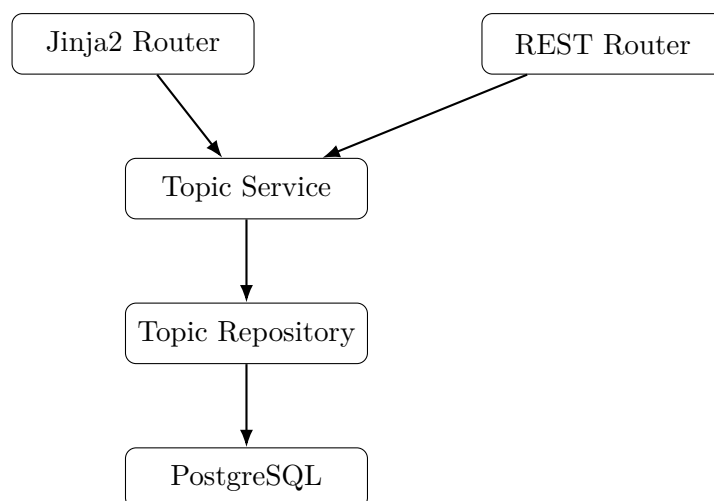
Only the `/app/*` namespace should belong to Flutter.

14 Shared Domain Logic

Consider a topic.

Both presentation layers may need topic data.

The architecture should be:



This avoids duplicate rules.

For example, slug validation, publication status, visibility rules, and topic retrieval should not be separately reimplemented in the Jinja router and the API router.

15 Authentication Strategy

The system has two browser-facing presentation layers, so authentication must be designed as a cross-frontend concern.

The project may use JWT-based authentication, but the final production mechanism should be selected with the browser architecture in mind.

If cookie-based browser authentication is selected, the architecture should include:

- secure cookies;
- appropriate `HttpOnly` settings;
- `Secure` in production;
- appropriate `SameSite` policy;
- CSRF protection for state-changing cookie-authenticated web/HTMX operations;
- server-side ownership checks;
- role-based authorization.

If bearer-token authentication is retained for Flutter, the token lifecycle and browser storage strategy must be explicitly designed.

The important architectural point is that authentication should not be duplicated as unrelated systems merely because Jinja2 and Flutter are different frontends.

16 SEO Strategy

Public pages should be designed as first-class HTML documents.

Each important public page should have, where applicable:

- unique title;
- meta description;
- canonical URL;
- semantic headings;

- meaningful body content;
- internal links;
- breadcrumbs;
- Open Graph metadata;
- structured data where appropriate;
- sitemap coverage;
- robots policy.

Example:

```
/topics/python-functions
```

should be rendered by Jinja2.

The authenticated Flutter area can be treated differently because it is application UI rather than the primary search-indexed content surface.

17 Static Assets

There are two types of static assets.

17.1 Normal Website Assets

These belong to:

```
backend/app/static/
```

Examples:

```
static/  
+-- css/  
+-- js/  
+-- images/
```

They can support Jinja2 and HTMX pages.

17.2 Flutter Assets

Compiled Flutter assets belong to:

```
backend/app/flutter/web/
```

They should be treated as generated deployment output rather than hand-edited source.

18 Flutter Build and Integration Process

During development, Flutter source is maintained normally.

The production sequence is:

1. Enter the Flutter project.
2. Run Flutter tests.
3. Build Flutter Web.
4. Obtain `build/web`.

5. Copy the generated web files into the backend's Flutter deployment directory.
6. Run backend tests.
7. Deploy the FastAPI application.

Conceptually:

```
frontend/flutter_app/
  |
  | flutter build web
  v
frontend/flutter_app/build/web/
  |
  | copy generated output
  v
backend/app/flutter/web/
  |
  | deploy
  v
FastAPI Cloud
```

19 Build Automation

Create:

```
scripts/build_flutter.py
```

The script should eventually perform:

1. verify that Flutter is available;
2. execute the Web build;
3. remove or replace stale generated Flutter output;
4. copy the new build output;
5. fail loudly if any step fails.

A later CI pipeline can perform the same sequence automatically.

A possible command concept is:

```
python scripts/build_flutter.py
```

The exact implementation can be introduced after the basic architecture is working.

20 Development Workflow

Development should remain simple.

20.1 Backend Development

Run FastAPI locally:

```
uv run uvicorn app.main:app --reload
```

20.2 Flutter Development

Run Flutter separately:

```
flutter run -d chrome
```

During development, Flutter can communicate with the local FastAPI API.
The developer does not need to rebuild Flutter for every Dart change.

20.3 Production

Production is different:

```
flutter build web
```

followed by integration of the generated files into the FastAPI deployment.

21 Database and Data Layer

PostgreSQL is the shared persistent data store.

The intended dependency direction is:

```
Jinja2 Router ----\  
HTMX Router -----+--> Service --> Repository --> PostgreSQL  
REST Router -----/  
WebSocket -----/
```

This allows one topic, progress, user, bookmark, or discussion implementation to serve multiple presentation interfaces.

Raw SQL can remain inside repository/query modules.

The repository layer should own:

- SQL statements;
- query parameters;
- database interaction;
- result mapping.

Services should own:

- business rules;
- workflow decisions;
- authorization-related domain checks;
- coordination between repositories.

22 API Design

The API should be versioned.

Example:

```
/api/v1/auth/login  
/api/v1/auth/register  
/api/v1/topics  
/api/v1/topics/{slug}  
/api/v1/progress  
/api/v1/bookmarks  
/api/v1/users/me
```

The API should return JSON.

The Flutter application should not directly access PostgreSQL.

The dependency remains:

```
Flutter -> HTTP API -> FastAPI -> Service -> Repository -> PostgreSQL
```

This preserves the backend boundary.

23 WebSocket Design

Real-time features can use:

```
/ws/*
```

Examples may include:

```
/ws/chat  
/ws/notifications
```

The WebSocket layer should still use shared services where business logic is required.

Do not place large amounts of database/business logic directly inside the WebSocket handler.

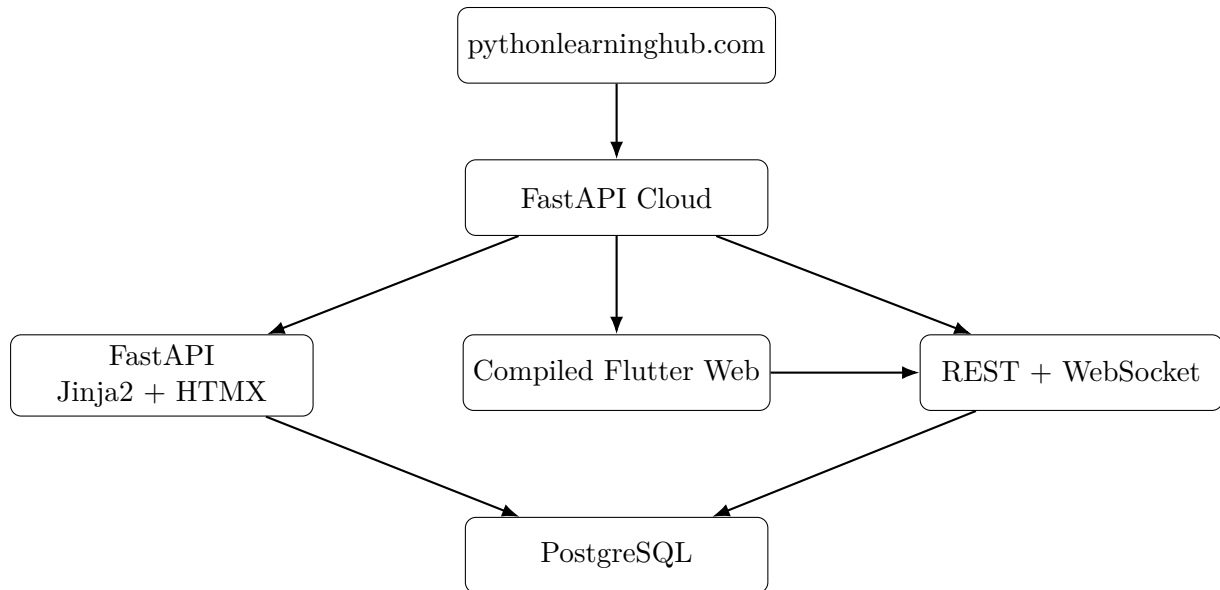
24 Security Boundaries

Security should be considered at every layer.

- Public pages should expose only intentionally public information.
- Authenticated API endpoints should verify authentication.
- User-specific resources should verify ownership.
- Administrative endpoints should verify roles.
- State-changing browser operations need CSRF protection when cookie authentication is used.
- Database queries must use parameterized values.
- Secrets must be stored in environment variables or a secure secret-management mechanism.
- Production debugging output should not reveal secrets or sensitive information.
- Rate limiting should be applied to sensitive endpoints where appropriate.
- File uploads should be validated and constrained.

25 Deployment Architecture

The initial deployment can be deliberately simple.



The key simplification is that the public browser-facing application can be deployed as one FastAPI application.

26 FastAPI Cloud Deployment Model

The deployment process should be understood as:

```

Git repository
|
v
FastAPI Cloud build
|
v
Python application + generated Flutter files
|
v
Running FastAPI application
|
v
Custom domain
  
```

The deployment must include the generated Flutter files.

The exact build configuration should be tested in the target FastAPI Cloud environment before treating the automated Flutter build process as final.

27 Custom Domain

The desired production experience is:

```
pythonlearninghub.com
```

rather than requiring users to visit separate frontend domains.

The custom domain points to the FastAPI deployment.

TLS should be enabled in production.

The final browser-visible routing remains:

```
/ -> Jinja2
/roadmap -> Jinja2
/topics/* -> Jinja2
/app/* -> Flutter
/api/v1/* -> JSON
/ws/* -> WebSocket
```

28 What Happens When the User Switches Frontends?

This should be explicitly understood.

28.1 Jinja2 to Flutter

Starting URL:

```
/topics/python-functions
```

A normal link:

```
<a href="/app/dashboard">Dashboard</a>
```

causes:

```
Browser
-> FastAPI
-> /app/*
-> Flutter
```

28.2 Flutter to Jinja2

Starting URL:

```
/app/dashboard
```

Flutter navigates the browser to:

```
/topics/python-functions
```

FastAPI sees the public route and returns Jinja2 HTML.

28.3 HTMX Within Jinja2

HTMX never needs to take ownership of Flutter routing.

It performs:

```
Jinja2 page
-> HTMX request
-> FastAPI
-> HTML fragment
-> DOM update
```

28.4 Flutter Internal Navigation

Flutter handles:

```
/app/dashboard  
/app/profile  
/app/progress  
/app/bookmarks
```

internally.

Thus:

Jinja2 and Flutter communicate through browser URLs; HTMX operates inside the Jinja2 world; Flutter Router operates inside the Flutter world.

29 Caching Strategy

Caching can be introduced gradually.

Potential cacheable content includes:

- public roadmap data;
- published topic data;
- category lists;
- popular searches;
- public metadata.

Redis can be added later if the application needs it.

Do not introduce Redis merely because the architecture diagram contains it. First measure the application's actual requirements.

30 Testing Strategy

Testing should cover each architectural boundary.

30.1 Repository Tests

Verify:

- SQL correctness;
- parameter handling;
- expected result mapping;
- not-found behavior.

30.2 Service Tests

Verify:

- business rules;
- authorization rules;
- workflows;
- edge cases.

30.3 API Tests

Verify:

- HTTP status codes;
- authentication;
- authorization;
- JSON response structures;
- validation;

30.4 Web Tests

Verify:

- public URLs return HTML;
- templates contain expected content;
- HTMX endpoints return fragments;
- redirects work.

30.5 Flutter Tests

Verify:

- routing;
- widgets;
- API integration;
- authentication state;
- error states.

30.6 End-to-End Tests

At least the following flows should eventually be tested:

1. visitor opens a public topic;
2. visitor searches;
3. visitor enters the Flutter application;
4. user logs in;
5. Flutter loads user data;
6. user updates progress;
7. user navigates back to public content;
8. public content remains directly accessible.

31 Development Phases

The project should be built incrementally.

31.1 Phase 1: Backend Foundation

Implement:

- FastAPI application;
- settings/configuration;
- database connection;
- repository structure;
- service structure;
- basic error handling;
- health endpoint.

Goal:

A clean FastAPI backend can run locally and connect to PostgreSQL.

31.2 Phase 2: Public Website

Implement:

- base template;
- home page;
- roadmap;
- topics;
- tutorials;
- search;
- SEO metadata;
- semantic HTML;
- internal linking.

Goal:

The platform is useful even if JavaScript is disabled.

31.3 Phase 3: HTMX

Add:

- search interactions;
- filtering;
- small progress interactions;
- comments/discussion interactions;
- partial page updates.

Goal:

Improve interactivity without turning the public site into a JavaScript-heavy application.

31.4 Phase 4: REST API

Implement:

- API versioning;
- authentication endpoints;
- user endpoints;
- topic endpoints;
- progress endpoints;
- bookmark endpoints.

Goal:

Flutter has a stable backend contract.

31.5 Phase 5: Flutter Application

Create the Flutter application.

Start with:

- application shell;
- routing;
- login;
- dashboard;
- profile;
- progress;
- bookmarks.

Goal:

Flutter can consume the REST API successfully.

31.6 Phase 6: Integrate Flutter With FastAPI

Build:

```
flutter build web
```

and integrate the generated files into:

```
backend/app/flutter/web/
```

Then test:

```
/app  
/app/dashboard  
/app/profile
```

Goal:

Flutter is accessible through the same domain as Jinja2.

31.7 Phase 7: Cross-Frontend Navigation

Implement and test:

```
Jinja2 -> Flutter  
Flutter -> Jinja2
```

Use ordinary browser URLs as the boundary.

Goal:

The user experiences one website rather than two separate sites.

31.8 Phase 8: Production Deployment

Configure:

- FastAPI Cloud;
- PostgreSQL;
- environment variables;
- custom domain;
- TLS;
- production logging;
- monitoring;
- deployment workflow.

32 Common Mistakes to Avoid

32.1 Mistake 1: Separate Domain for Every Frontend

Avoid unnecessary complexity such as:

```
www.example.com  
app.example.com  
api.example.com  
flutter.example.com
```

The initial design should use one domain unless a real requirement justifies separation.

32.2 Mistake 2: Global Flutter Catch-All

Do not make:

```
ANY URL -> Flutter index.html
```

Instead:

```
/app/* -> Flutter
```

Only.

32.3 Mistake 3: Business Logic in Routers

Do not duplicate database and business rules across:

```
Jinja2 routers
API routers
Flutter-specific endpoints
```

Use services and repositories.

32.4 Mistake 4: HTMX as a Replacement for Flutter

HTMX and Flutter have different responsibilities.

HTMX is for small server-rendered interactions.

Flutter is for the application.

32.5 Mistake 5: Flutter Directly Accessing the Database

Never expose PostgreSQL directly to Flutter.

The correct path is:

```
Flutter -> FastAPI API -> Service -> Repository -> PostgreSQL
```

32.6 Mistake 6: Treating Generated Flutter Files as Source

Do not manually edit:

```
main.dart.js
flutter.js
```

The source of truth remains the Flutter project.

33 Production Directory Model

After a successful build, the backend portion may conceptually look like:

```
backend/
|
+-- app/
|   +-- main.py
|   +-- routers/
|   +-- services/
|   +-- repositories/
|   +-- schemas/
|   +-- templates/
|   +-- static/
|   |
|   +-- flutter/
|       +-- web/
|           +-- index.html
|           +-- main.dart.js
|           +-- flutter.js
|           +-- assets/
|           +-- canvaskit/
|
+-- pyproject.toml
```

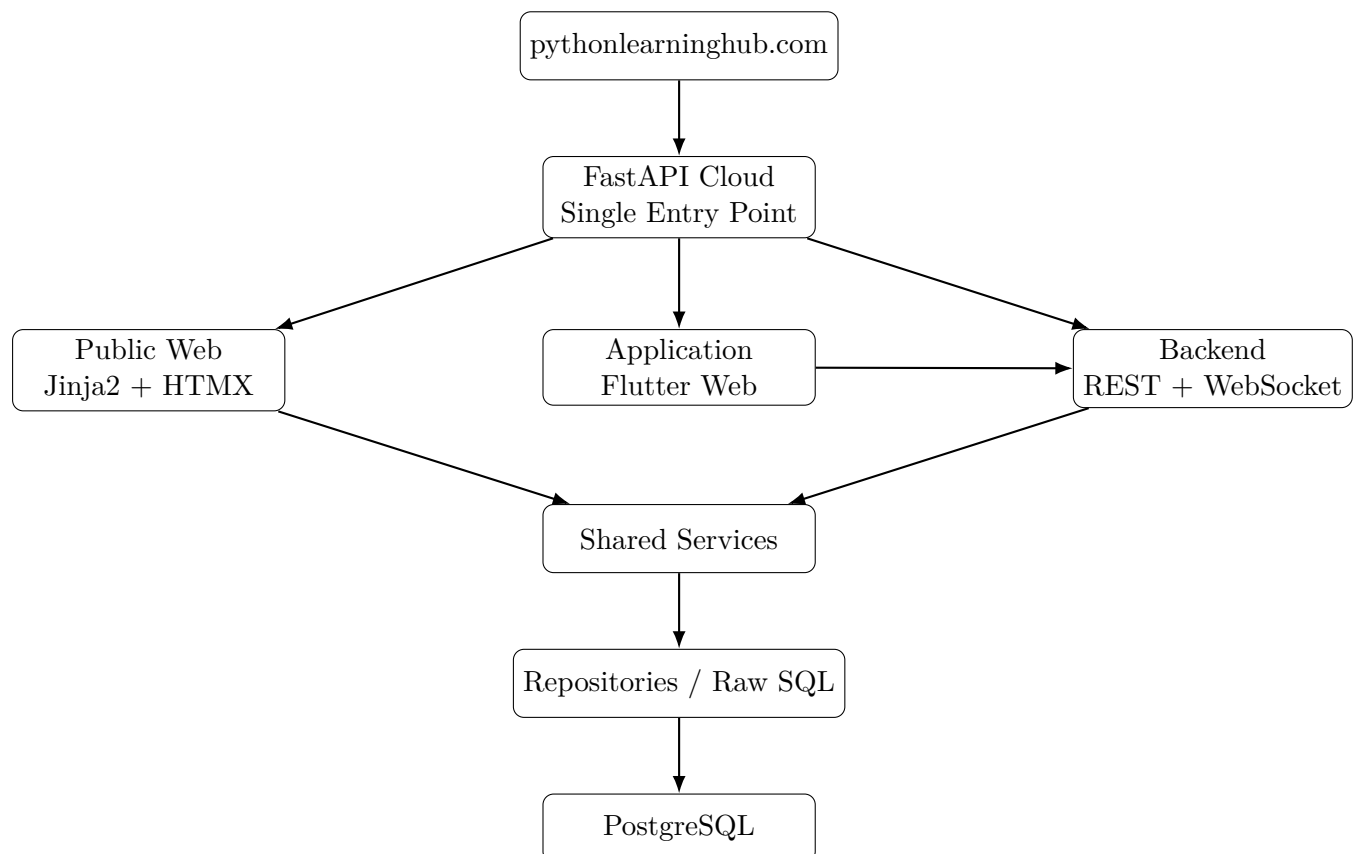
FastAPI sees:

```
/app/*
```

and serves the compiled frontend.

34 Final Architecture

The final conceptual architecture is:



The final URL model is:

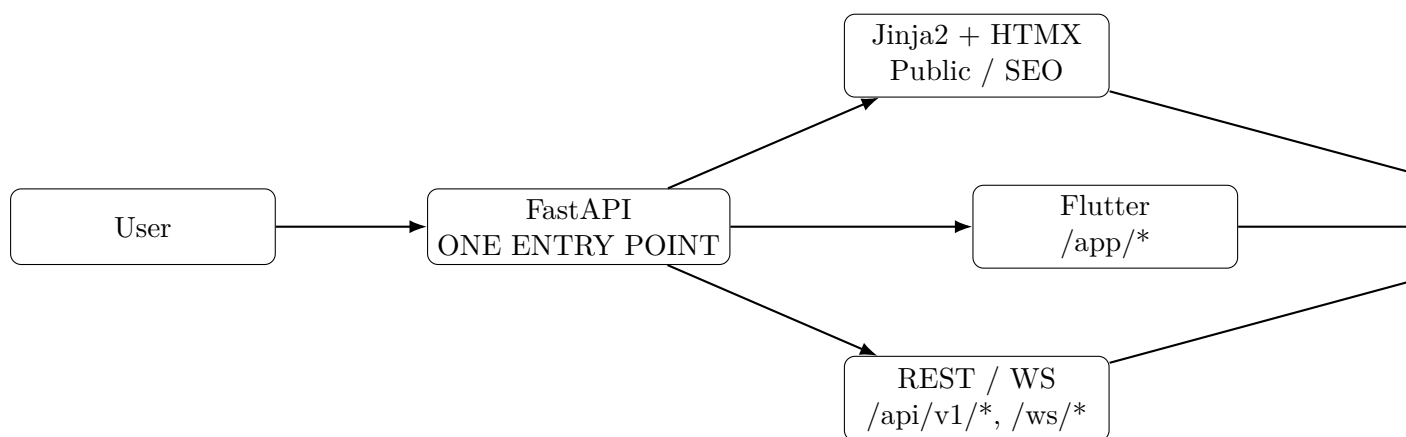
/	-> Jinja2
/roadmap	-> Jinja2
/topics/*	-> Jinja2
/tutorials/*	-> Jinja2
/guides/*	-> Jinja2
/search	-> Jinja2
/web-components/*	-> HTMX
/app/*	-> Flutter Web
/api/v1/*	-> JSON API
/ws/*	-> WebSocket

35 Implementation Checklist

#	Task	Done
1	Create backend FastAPI project structure	
2	Configure PostgreSQL	
3	Create repository layer	
4	Create service layer	
5	Create public Jinja2 pages	
6	Create HTMX component routes	
7	Implement SEO metadata	
8	Implement REST API v1	
9	Implement authentication	
10	Create Flutter Web application	
11	Implement Flutter routing under /app	
12	Implement Flutter API client	
13	Implement Flutter authentication flow	
14	Build Flutter Web	
15	Copy Flutter build into backend deployment directory	
16	Implement /app/* fallback correctly	
17	Test public routes	
18	Test Flutter deep links	
19	Test Flutter API calls	
20	Test Jinja2-to-Flutter navigation	
21	Test Flutter-to-Jinja2 navigation	
22	Configure production environment variables	
23	Deploy FastAPI Cloud application	
24	Configure custom domain	
25	Verify TLS/HTTPS	
26	Verify production SEO routes	
27	Add automated Flutter build process	
28	Add automated tests and CI/CD	
29	Add monitoring and error reporting	
30	Document the complete deployment process	

36 Final Mental Model

The entire project can be remembered with one diagram:



The central rule is:

One domain → one FastAPI entry point → multiple presentation layers → shared application logic → one data layer.

This lets Python Learning Hub remain one coherent product while using the technology that best fits each part of the user experience.